Лабораторная работа 2

Счётчик калорий

1 Диаграмма классов

Диаграмма классов описывает программную структуру системы «Счётчик калорий», по которой написан код на C#. Имена классов, полей и операций записаны на английском языке, поэтому код совпадает с диаграммой буквально. Классы соответствуют сущностям модели предметной области: каталог продуктов (ProductCatalog), дневник питания (FoodDiary), порция (Portion), продукт (Product), ингредиент (Ingredient) и приготовленная партия (CookedBatch). Сценарий учёта организует обработчик учёта порции (PortionHandler). Сущность «Пользователь» классом не стала: программа ведёт один дневник, поэтому хранить пользователя не требуется.

Продукт и приготовленная партия обобщены абстрактным классом еды (Food). Еда имеет название и защищённый конструктор, который вызывают только наследники. Абстрактная операция GetKcalPer100g возвращает калорийность на 100 г: продукт берёт её с этикетки, а партия вычисляет по общим калориям и готовой массе. Операция CalculateKcal одна для всех видов еды и пересчитывает калорийность на заданную массу. Операция CanServe проверяет, можно ли подать порцию заданной массы; партия переопределяет её, чтобы масса порции не превышала готовую массу. Благодаря этому порцию можно взять и из партии, и непосредственно из продукта, а отдельный калькулятор калорий, показанный на диаграмме коммуникаций, при проектировании не потребовался: расчёт распределён между классами еды.

На диаграмме использованы пять видов связей. Направленные ассоциации показывают, какие объекты знает каждый класс: обработчик связан с каталогом, дневником и текущим предпросмотром порции, порция — со своей едой, ингредиент — с продуктом. Агрегации связывают каталог с продуктами и дневник с порциями: части существуют независимо от целого. Композиция связывает партию с ингредиентами, которые создаются внутри партии и без неё не существуют. Обобщение связывает продукт и партию с едой. Зависимости показывают классы, которые используются только как параметры или локальные переменные операций; партию обработчик создаёт сам, что отмечено стереотипом «create».

Калории фиксируются при создании объектов: общие калории партии накапливаются при добавлении ингредиентов, калории порции рассчитываются в её конструкторе, и последующие изменения каталога их не меняют. Операция Preview может

не сформировать порцию, поэтому у её результата указана кратность [0..1]. Классы, их члены и связи представлены на рисунке 1, описание диаграммы приведено в таблице 1.

Таблица 1 – Описание диаграммы классов

Поле	Описание
Вопрос	Из каких классов состоит программа учёта порций и как эти классы связаны?
Читатель	Разработчик
Что сделает читатель	Напишет каркас кода: классы, поля, свойства, конструкторы и заголовки операций
Что показано	– классы с атрибутами и операциями, их видимостью и типами — этого достаточно для каркаса кода; – абстрактный класс Food с двумя наследниками — порция одинаково рассчитывается из продукта и из партии; – виды связей и кратности — они определяют поля-ссылки и коллекции; – зависимости — классы, которые нужны только внутри операций.
Что не показано	– порядок вызовов внутри операций — он показан на диаграммах последовательностей; – хранение данных между запусками программы; – пользовательский интерфейс и обработка ошибок ввода.
Допущения	– формулы расчёта калорий на диаграмме не записаны и берутся из предметной области; – типы Real, String, Boolean и DateTime реализуются типами double, string, bool и DateTime; – роль на конце ассоциации становится закрытым полем, а кратность 0..* или 1..* — списком List; – кратность [0..1] у результата Preview реализуется типом Portion?, допускающим значение null.

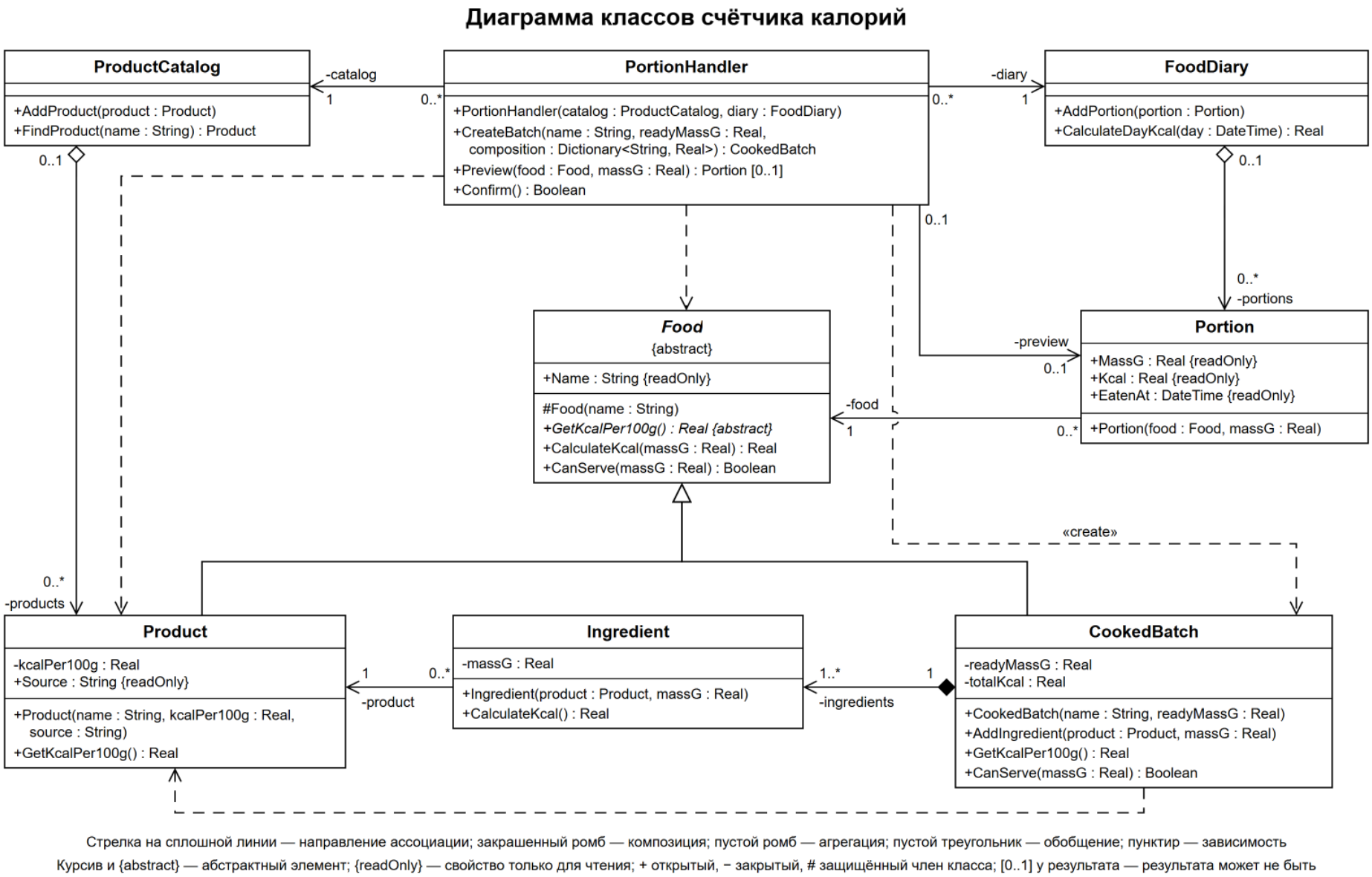


Рисунок 1 – Диаграмма классов счётчика калорий

2 Диаграмма последовательностей операции CreateBatch

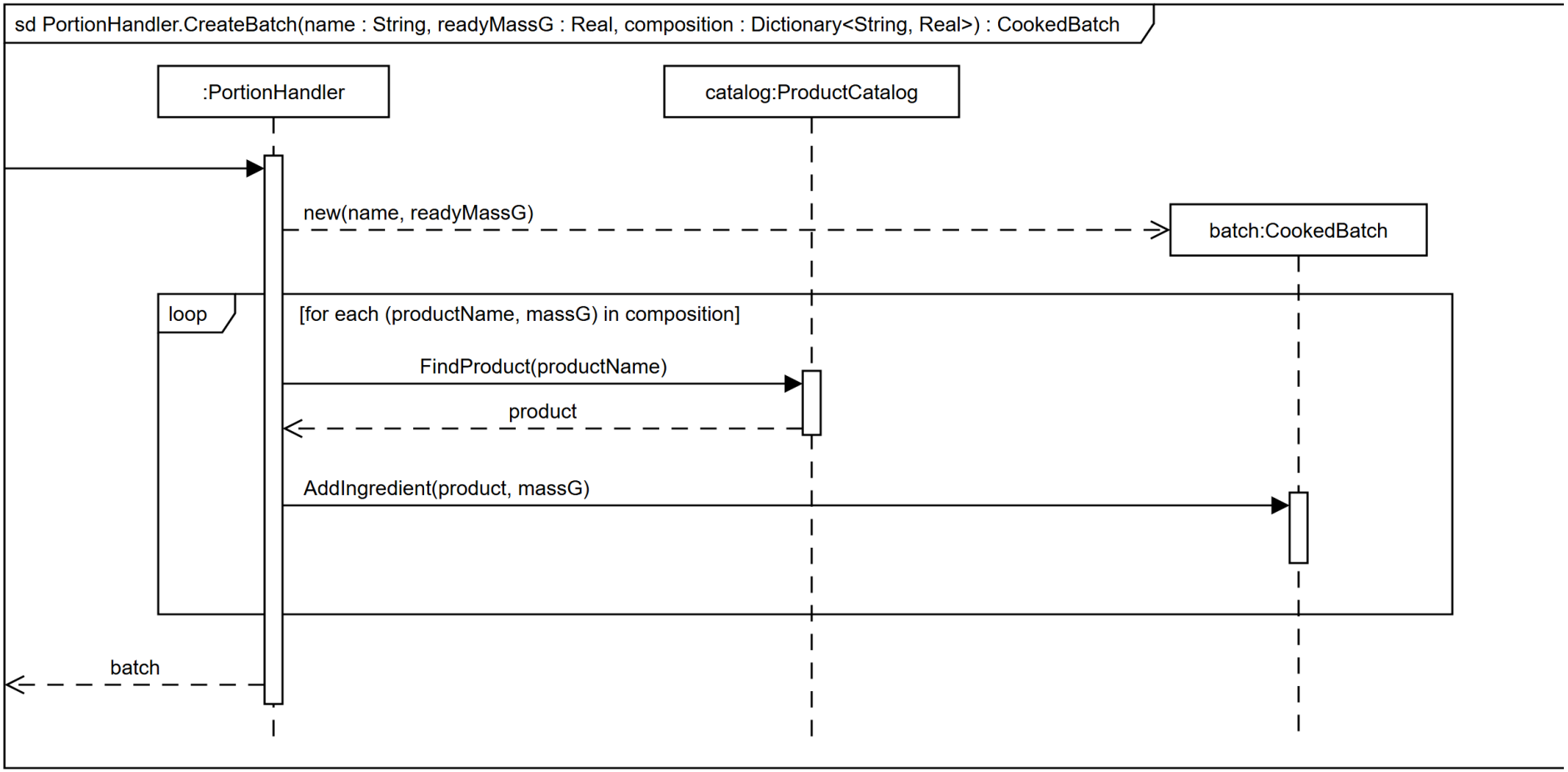
Диаграмма описывает операцию CreateBatch обработчика учёта порции, которая создаёт приготовленную партию по названию, готовой массе и составу. Состав передаётся словарём: ключ — название продукта из каталога, значение — его масса в граммах. Во взаимодействии участвуют обработчик, каталог продуктов catalog и создаваемая партия batch. Диаграмма демонстрирует фрейм цикла.

Сначала обработчик создаёт пустую партию сообщением new(name, readyMassG). Блок партии расположен ниже остальных, так как объект появляется только в этот момент. Затем во фрейме loop для каждой пары «название — масса» обработчик находит продукт в каталоге, сохраняет его в переменную product и передаёт партии вместе с массой операцией AddIngredient; её устройство показано на отдельной диаграмме. После цикла партия возвращается вызывающему объекту. В учебном примере цикл выполняется дважды: для 40 г хлопьев и для 200 г молока. Взаимодействие представлено на рисунке 2, описание диаграммы — в таблице 2.

Таблица 2 – Описание диаграммы последовательностей операции CreateBatch

Поле	Описание
Вопрос	Как обработчик собирает приготовленную партию из введённого состава?
Читатель	Разработчик
Что сделает читатель	Напишет тело операции CreateBatch
Что показано	– партия создаётся до цикла, и ингредиенты добавляются в уже существующий объект; – фрейм loop по составу — число ингредиентов заранее неизвестно; – продукт берётся из каталога, поэтому партия получает объект продукта из каталога, а не его копию; – созданная партия возвращается вызывающему объекту.
Что не показано	– что происходит внутри AddIngredient — это показано на рисунке 4; – как пользователь вводит состав и массы; – реакция на пустой состав.
Допущения	– если названия нет в каталоге, FindProduct выбрасывает исключение KeyNotFoundException; на диаграмме это не изображено, так как продукты выбираются из каталога и такая ситуация считается ошибкой программы; – пара (productName, massG) — элемент словаря Dictionary<string, double>, в коде цикл записывается оператором foreach с разложением пары; – кратность 1..* у ингредиентов операция не проверяет: предполагается, что состав непустой.

Диаграмма последовательностей операции CreateBatch



Сплошная стрелка — вызов операции; пунктирная — возврат (подписан переменной, куда сохраняется результат) или создание объекта new(...)
Узкий прямоугольник — активность (выполнение операции); loop — фрейм цикла: сообщения внутри повторяются для каждой пары из composition

Рисунок 2 – Диаграмма последовательностей операции CreateBatch

3 Диаграмма последовательностей операции Preview

Диаграмма описывает операцию Preview обработчика учёта порции, которая формирует предпросмотр порции заданной массы из еды — продукта или приготовленной партии. Во взаимодействии участвуют обработчик, еда food и создаваемая порция preview. Участник food имеет абстрактный тип Food, поэтому то, какая реализация GetKcalPer100g будет выполнена, определяется во время работы программы. Диаграмма демонстрирует вложенные активности.

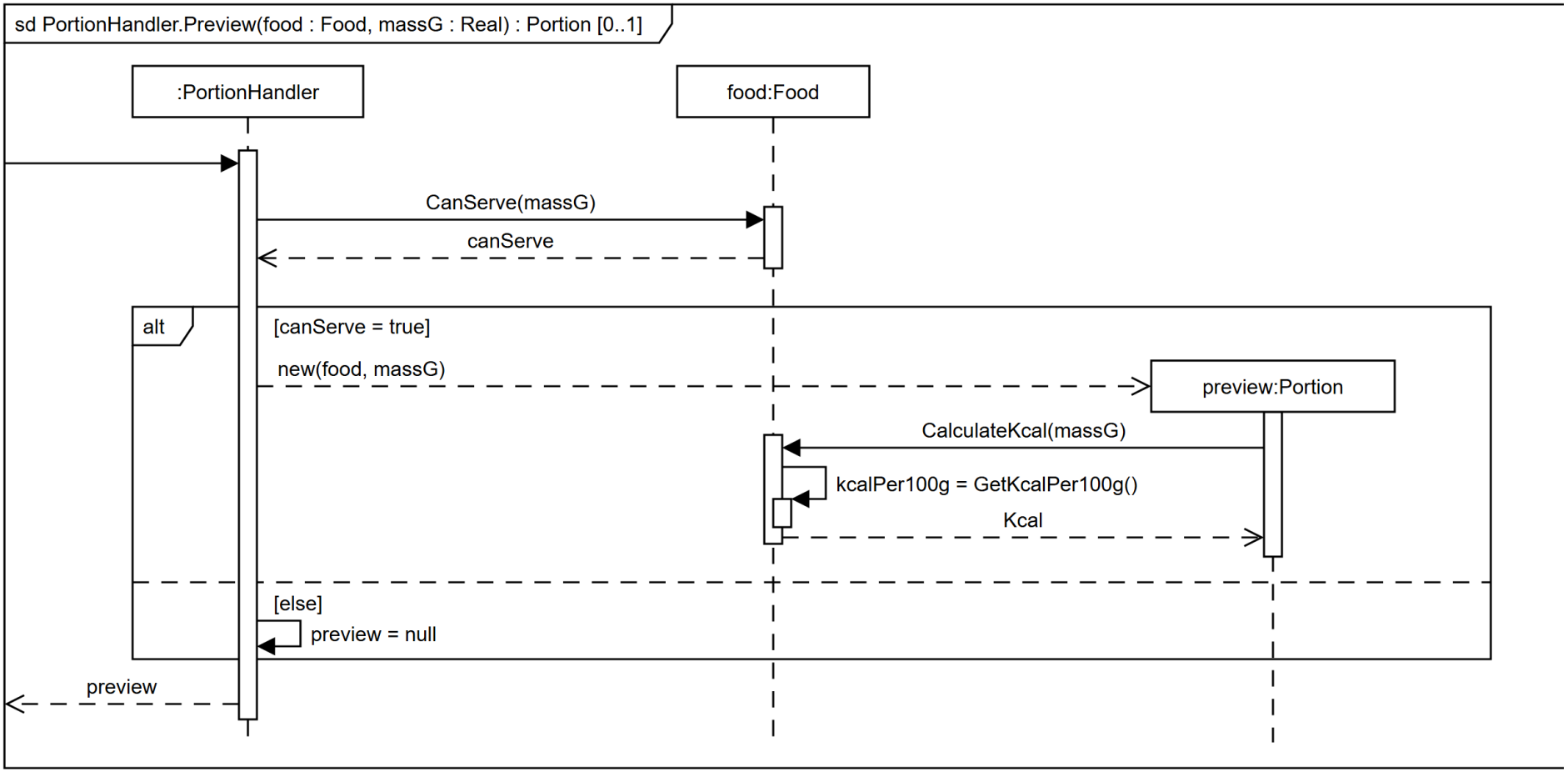
Обработчик спрашивает у еды, можно ли подать порцию такой массы, и сохраняет ответ в переменную canServe. Далее фрейм alt разделяет два случая. Если порцию подать можно, создаётся объект порции. Её конструктор вызывает у еды операцию CalculateKcal(massG), а эта операция — собственную операцию GetKcalPer100g. Активность самовызова вложена в активность CalculateKcal, которая выполняется во время активности конструктора порции. Результат сохраняется в свойство Kcal. В ветви [else] предпросмотр сбрасывается в null, чтобы ранее сформированный предпросмотр нельзя было подтвердить. Операция возвращает предпросмотр либо null, поэтому у её результата указана кратность [0..1].

В учебном примере порция массой 110 г из каши даёт 122,4 ккал, а запрос порции массой 300 г возвращает null, так как превышает готовую массу 220 г. Взаимодействие представлено на рисунке 3, описание диаграммы — в таблице 3.

Таблица 3 – Описание диаграммы последовательностей операции Preview

Поле	Описание
Вопрос	Как обработчик формирует предпросмотр порции и рассчитывает её калории?
Читатель	Разработчик
Что сделает читатель	Напишет операцию Preview, конструктор порции и операцию CalculateKcal класса Food
Что показано	– проверка CanServe выполняется до создания порции, поэтому порция с недопустимой массой не появляется; – вложенные активности конструктор порции → CalculateKcal → GetKcalPer100g: калории рассчитываются один раз при создании порции; – ветвь [else] сбрасывает прежний предпросмотр.
Что не показано	– правила CanServe для продукта и для партии; – подтверждение предпросмотра и запись порции в дневник (операция Confirm); – вывод предпросмотра пользователю.
Допущения	– формула калорий «калорийность на 100 г × масса / 100» на диаграмме не изображена; – конкретный объект food — продукт или партия; на диаграмме указан их общий абстрактный тип; – кратность [0..1] у результата реализуется типом Portion?.

Диаграмма последовательностей операции Preview



Сплошная стрелка — вызов операции; пунктирная — возврат (подписан переменной, куда сохраняется результат) или создание объекта new(...)

Узкий прямоугольник — активность, смещённый вправо — вложенная активность самовызова; alt — фрейм условия с ветвями

Рисунок 3 – Диаграмма последовательностей операции Preview

4 Диаграмма последовательностей операции AddIngredient

Диаграмма описывает операцию AddIngredient приготовленной партии, которая добавляет в партию ингредиент — массу конкретного продукта. Во взаимодействии участвуют партия, создаваемый ингредиент ingredient и продукт product. Диаграмма демонстрирует создание объекта.

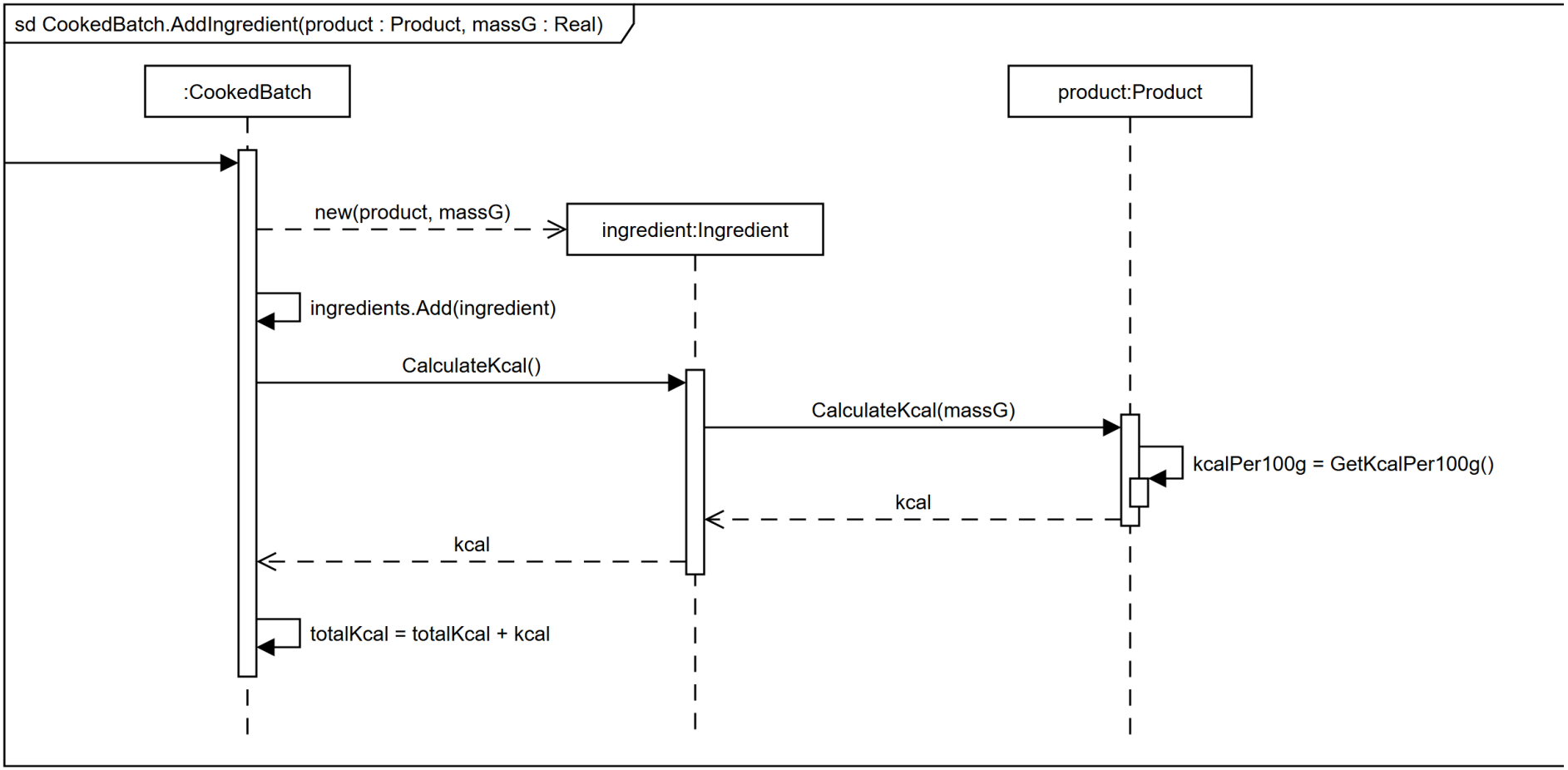
Партия создаёт ингредиент сообщением new(product, massG): ингредиент является частью партии по композиции, поэтому его создаёт сама партия. Созданный ингредиент добавляется в список ingredients. Затем партия запрашивает у ингредиента калории. Ингредиент вызывает у продукта операцию CalculateKcal(massG), а продукт — собственную операцию GetKcalPer100g. Результат возвращается в переменную kcal и прибавляется к общим калориям партии totalKcal. Действия внутри операции, не являющиеся вызовами, изображены петлёй без активности.

Общие калории партии накапливаются при добавлении ингредиентов и далее не пересчитываются. В учебном примере после двух вызовов они составляют 140,8 + 104 = 244,8 ккал. Взаимодействие представлено на рисунке 4, описание диаграммы — в таблице 4.

Таблица 4 – Описание диаграммы последовательностей операции AddIngredient

Поле	Описание
Вопрос	Как партия добавляет ингредиент и учитывает его калории?
Читатель	Разработчик
Что сделает читатель	Напишет операции AddIngredient и CalculateKcal класса Ingredient
Что показано	– создание ингредиента внутри партии — следствие композиции; – добавление ингредиента в список партии; – расчёт калорий по цепочке партия → ингредиент → продукт; – накопление общих калорий в поле totalKcal.
Что не показано	– пересчёт общих калорий на 100 г готовой массы (операция GetKcalPer100g партии); – удаление и изменение ингредиентов — программа их не поддерживает; – проверка массы ингредиента.
Допущения	– петли ingredients.Add(ingredient) и totalKcal = totalKcal + kcal обозначают операторы внутри метода, а не вызовы операций; – возврат из операции без результата (void) не изображён; – формула калорий продукта не изображена.

Диаграмма последовательностей операции AddIngredient



Сплошная стрелка — вызов операции; пунктирная — возврат (подписан переменной, куда сохраняется результат) или создание объекта new(...)

Узкий прямоугольник — активность, смещённый вправо — вложенная активность самовызова; петля без активности — действие внутри операции

Рисунок 4 – Диаграмма последовательностей операции AddIngredient

5 Диаграмма объектов

Диаграмма объектов показывает состояние программы в учебном примере после подтверждения порции. Такое же состояние получается при запуске программы. Каталог содержит продукты «Хлопья» (352 ккал на 100 г) и «Молоко» (52 ккал на 100 г). Каша содержит два ингредиента — 40 г хлопьев и 200 г молока, её готовая масса 220 г, общие калории 244,8 ккал. Дневник содержит порцию каши массой 110 г и калорийностью 122,4 ккал. Обработчик связан с каталогом и дневником.

Имена объектов подчёркнуты и записаны в виде «имя : Класс», под чертой указаны значения полей, включая унаследованное поле Name. Связи являются экземплярами ассоциаций, агрегаций и композиции диаграммы классов; у их концов подписаны роли. Диаграмма показывает значимую связь: ингредиенты ссылаются на те же объекты продуктов, что хранятся в каталоге, а порция — на партию, а не на продукты. Связи обработчика с предпросмотром нет, так как после подтверждения предпросмотр сброшен. Диаграмма представлена на рисунке 5, описание — в таблице 5.

Таблица 5 – Описание диаграммы объектов

Поле	Описание
Вопрос	Какие объекты и связи существуют после записи порции каши в дневник?
Читатель	Тестировщик
Что сделает читатель	Сверит значения полей и связи объектов в тестах и при отладке с ожидаемыми
Что показано	– значения полей учебного примера — эталон для проверки расчётов; – общие объекты продуктов у каталога и ингредиентов; – связь порции с партией, а не с продуктами; – отсутствие связи с предпросмотром после подтверждения.
Что не показано	– промежуточные состояния, например до подтверждения, когда порция связана с обработчиком, а дневник пуст; – порции за другие дни; – иерархия наследования классов.
Допущения	– время EatenAt условное: в программе оно берётся из часов компьютера в момент создания порции; – дробные значения записаны с точкой, как в коде.

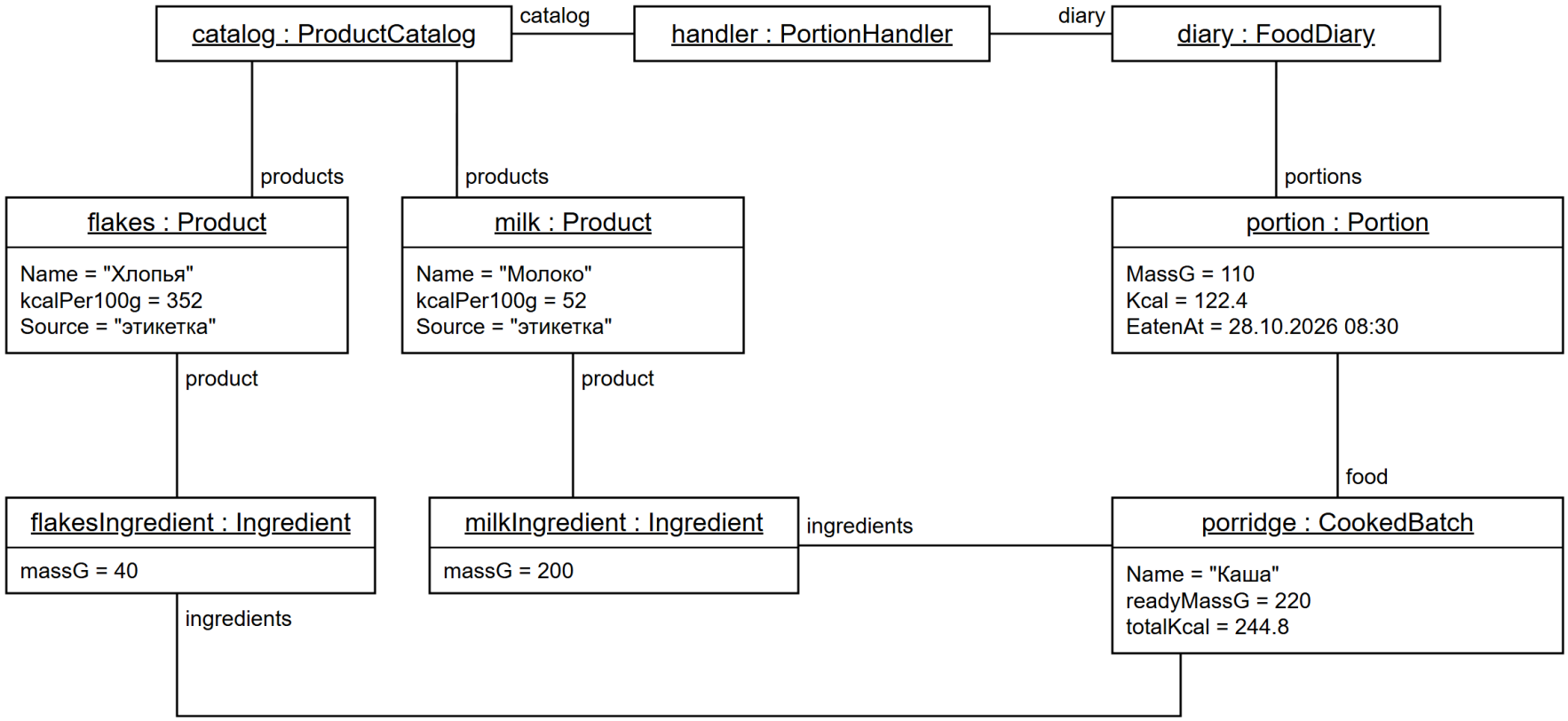


Рисунок 5 – Диаграмма объектов после подтверждения порции каши

6 Программный код

Код написан на языке C# для платформы .NET 10 и находится в решении Counter/CalorieCounter.slnx. Консольный проект CalorieCounter содержит классы диаграммы классов, каждый в отдельном файле пространства имён CalorieCounter. Проект CalorieCounter.Tests содержит автоматические тесты xUnit.

Код получен прямым проектированием по диаграммам: каждому элементу диаграммы соответствует определённая конструкция языка. Соответствие приведено в таблице 6.

Таблица 6 – Соответствие элементов диаграмм и кода

Элемент диаграммы	**Конструкция C#**
Закрытый атрибут -massG : Real	закрытое поле private double massG
Свойство +Name : String {readOnly}	свойство только для чтения public string Name { get; }
Роль на конце ассоциации с кратностью 1 или 0..1	поле-ссылка; при 0..1 — тип, допускающий null
Роль с кратностью 0..* или 1..*	поле-список List<T>
Защищённый конструктор #Food(name : String)	конструктор protected Food(string name)
Абстрактная операция	абстрактный метод без тела
Операция, повторённая в наследнике	метод virtual в предке и override в наследнике
Обобщение	наследование class Product : Food с вызовом конструктора предка : base(name)
Зависимость	параметр или локальная переменная операции
Сообщение M(x) с возвратом r	вызов с сохранением результата: Тип r = obj.M(x);
Создание объекта new(a, b)	T obj = new T(a, b);
Фрейм loop	цикл foreach
Фрейм alt	условный оператор if … else

Метод Main воспроизводит учебный пример: создаёт каталог с двумя продуктами, собирает кашу, запрашивает предпросмотр порций массой 300 г и 110 г и подтверждает вторую. Программа выводит калорийность каши 111,3 ккал на 100 г, отказ для порции 300 г, предпросмотр порции 110 г на 122,4 ккал и итог за день 122,4 ккал. Тесты проверяют расчёты для учебного примера и правила предпросмотра и подтверждения; все 26 тестов проходят.